

Information and Network Security

Study Guide

Dr. Amitava Sen
CSE, PIT
Parul University

7.1Hash

Function.....

3

7.1 Hash Function

What is a Hash Function?

- A hash function takes an input of any length and produces a fixed-length output (the hash value).
- It's designed to be one-way, meaning it's computationally infeasible to reverse the process and find the original input from the hash value.
- A good hash function should be efficient, meaning it can compute the hash value quickly, and ideally, it should minimize collisions, where two different inputs produce the same hash value.

- Key Properties of a Hash Function:
- Deterministic: Given the same input, the hash function always produces the same output.
- One-way: It's computationally infeasible to find the original input from the hash value.
- Collision resistance: It's difficult to find two different inputs that produce the same hash value.
- Uniform distribution: The hash function should distribute the hash values evenly over the output range to avoid clustering and potential performance issues.

Applications of Hash Functions:

- Data Integrity Checking: Hash functions can be used to verify if data has been tampered with. By comparing the hash value of the original data with the hash value of the received data, one can detect any changes.
- Indexing: In hash tables, a hash function is used to map keys to indices in an array, enabling fast retrieval of data based on the key.
- Cryptography:
- Digital Signatures: Hash functions are used to create a unique "fingerprint" of a message, which is then encrypted using the signer's private key. This allows for verifying the authenticity and integrity of the message using the signer's public key.
- Password Storage: Instead of storing passwords in plain text, websites store the hash value of the password, which provides a level of security as it's difficult to retrieve the original password from the hash.

Message Authentication Codes (MACs): MACs use a shared secret key along with a hash function to ensure the integrity and authenticity of messages.

Other Applications:

- **Fingerprinting:** Hash values can be used to identify files or data and detect duplicates.
- **Bloom Filters:** These probabilistic data structures use hash functions to efficiently check for the presence of elements in a set.
- **Network Routing:** Hashing is used in load balancing algorithms to distribute requests to servers.
- **Blockchain:** Hashing is used in proof-of-work algorithms to secure the integrity of the blockchain.

7.2 Security Requirements for Cryptographic Hash Functions

Cryptographic hash functions must be secure to prevent various attacks. They need to be collision-resistant, meaning it's difficult to find two different inputs that produce the same hash output. Additionally, they should be preimage resistant, meaning it's computationally infeasible to find an input that produces a given hash output. Finally, they should be second-preimage resistant, meaning it's hard to find a second input that produces the same hash as a given input.

- **Collision Resistance:**
A collision occurs when two different inputs produce the same hash value. This can be exploited by attackers to forge signatures or modify data without detection, according to a blog post on Trio MDM. A collision-resistant hash function makes it computationally infeasible to find such colliding inputs.
- **Preimage Resistance:**
This property ensures that given a hash value, it's computationally infeasible to find the original input that generated that hash. This is crucial for preventing attackers from reversing the hashing process and potentially recovering sensitive information.
- **Second-Preimage Resistance:**
This is a stronger form of preimage resistance. It requires that it's computationally infeasible to find a second input that produces the same hash as a given input. This is particularly important for digital signatures, where an attacker might try to find a different message with the same hash as a valid signed message.
- **Other Security Considerations:**
 - Determinism:** The same input should always produce the same hash output.
 - Efficiency:** Hash functions should be computationally efficient, meaning they can be quickly calculated.
 - Avalanche Effect:** Small changes in the input should result in significant, seemingly random changes in the output hash. This helps

ensure that small changes in the input don't create predictable changes in the hash.

Security against known attacks: Hash functions should be designed to withstand various cryptanalytic attacks.

7.3 Hash Functions Based on Cipher BlockChaining

Hash functions based on Cipher Block Chaining (CBC) utilize a compression function, f , that takes a chaining variable (the previous hash) and a block of data, producing a new chaining variable. This iterative process is repeated for each data block, with the final chaining variable being the hash value of the entire input.

A common example of a hash function used in blockchain based on cipher block chaining (CBC) is SHA-256

Hash Function based on cipher block chaining

Two major categories of hash functions are: *dedicated hash functions* and *block cipher-based hash function*.

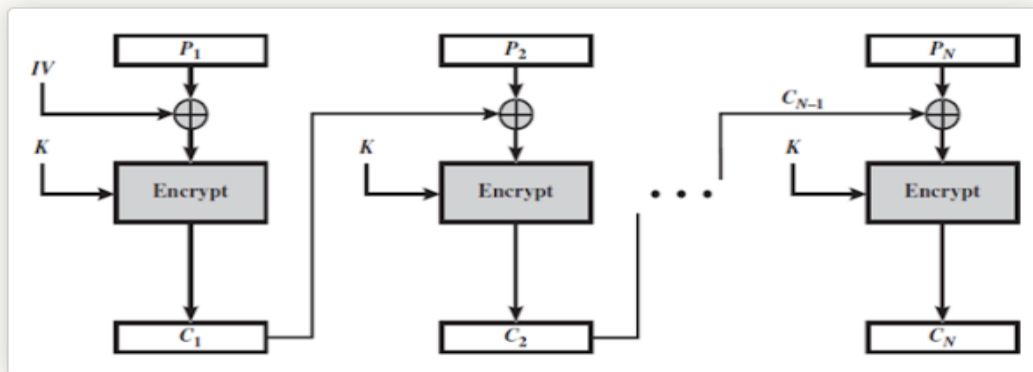


Figure: Cipher Block Chaining Mode

Block cipher is a popular encryption-decryption primitives. To encrypt, the block cipher accepts a key K and a plain text block P as input and produces a cipher text block $C = E(K, P)$, also written as $C = E_K(P)$.

7.4 Secure Hash Algorithm (SHA).

Secure Hash Algorithm (SHA), is a family of cryptographic hash functions published by the National Institute of Standards and Technology (NIST). These algorithms are used to generate a fixed-size "fingerprint" (hash) of a message, ensuring data integrity and authenticity. While SHA-1 was previously widely used, it's now considered weak due to vulnerabilities. SHA-2 and SHA-3 offer more robust security and are more commonly used today.

Key Features of SHA:

Hashing: SHA algorithms transform data into a fixed-size hash value (also called a message digest).

One-way function: It's computationally infeasible to reverse the hashing process and recover the original data from its hash.

Data Integrity: Any change to the original data will result in a different hash, allowing for verification of data integrity.

Applications:

SHA algorithms are used in various applications, including:

Digital signatures

Data authentication

Password hashing

Secure communications (TLS/SSL)

Types:

SHA-1: An older algorithm, now considered weak.

SHA-2: A family of algorithms with different digest sizes (e.g., SHA-256, SHA-512), offering better security than SHA-1.

SHA-3: The latest algorithm, using a different internal structure for enhanced security.

How SHA works (general principle):

Input: The algorithm takes an input message of any length.

Padding: The input message is padded to ensure its length is a multiple of a specific block size.

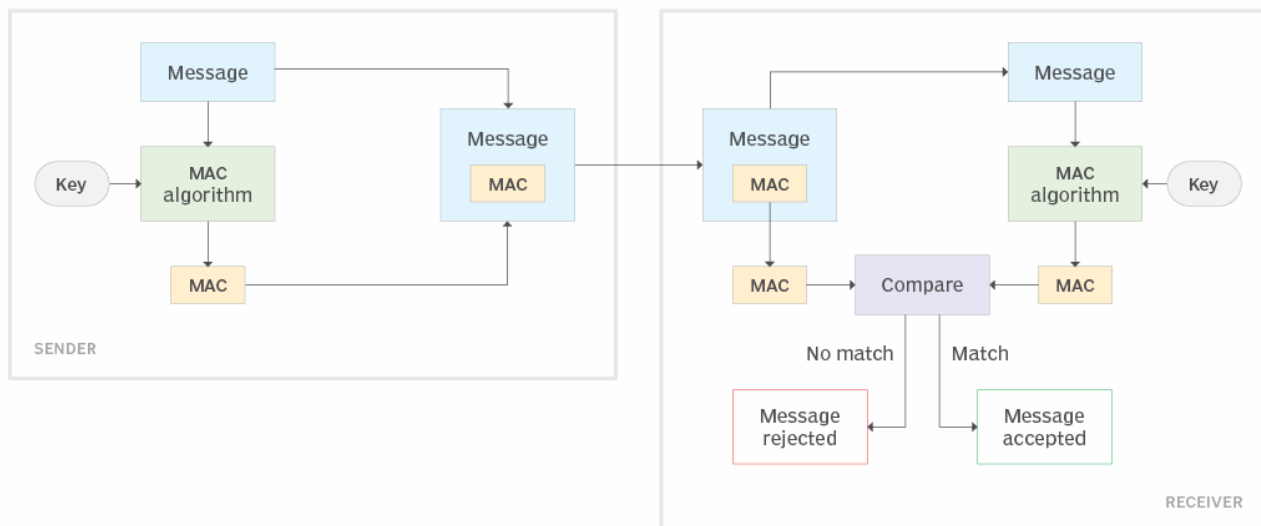
Compression Function: The padded message is processed through a compression function, which iteratively transforms the input into a hash value.

Output: The final hash value, a fixed-size string, is produced

7.5 MAC: Message Authentication Requirements

A message authentication code (MAC) is a cryptographic checksum applied to a message in network communication to guarantee its integrity and authenticity. A MAC ensures the transmitted message originated with the stated sender and was not modified during transmission, either accidentally or intentionally. A MAC is sometimes referred to as a tag because of the way it is added to the message it is verifying.

Generating and verifying message authentication codes



How does MAC provide authentication?

The MAC algorithm then generates authentication tags of a fixed length by processing the message. The resulting computation is the message's MAC. This MAC is then appended to the message and transmitted to the receiver. The receiver computes the MAC using the same algorithm.

7.6 MAC: Message Authentication Functions.

Message authentication functions ensure the integrity and authenticity of messages. These functions are crucial in digital communication and security. They operate at two levels and can be grouped into classes, including message encryption, message authentication codes (MACs), and digital signatures. MACs use a secret key, while digital signatures use cryptographic keys for authentication. These functions ensure that a message hasn't been altered during transmission and confirm the sender's identity.

All message authentication and digital signature mechanisms are based on two functionality levels:

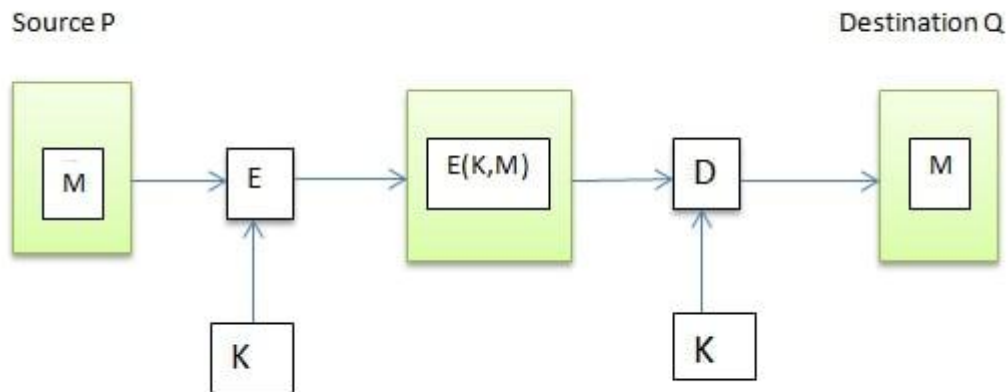
Lower level: At this level, there is a need for a function that produces an authenticator, which is the value that will further help in the authentication of a message.

Higher-level: The lower level function is used here in order to help receivers verify the authenticity of messages.

These message authentication functions are divided into three classes:

Message encryption: While sending data over the internet, there is always a risk of a Man in the middle(MITM) attack. A possible solution for this is to use message encryption. In message encryption, the data is first converted to a ciphertext and then sent any further. Message encryption can be done in two ways:

Symmetric Encryption: Say we have to send the message M from a source P to destination Q . This message M can be encrypted using a secret key K that both P and Q share. Without this key K , no other person can get the plain text from the ciphertext. This maintains confidentiality. Further, Q can be sure that P has sent the message. This is because other than Q , P is the only party who possesses the key K and thus the ciphertext can be decrypted only by Q and no one else. This maintains authenticity. At a very basic level, symmetric encryption looks like this:



7.7 Requirements for Message Authentication Codes.

Message Authentication Codes (MACs) have specific requirements to ensure the integrity and authenticity of messages. These requirements revolve around the ability to detect any tampering or alterations to the message during transmission, as well as verifying the source of the message. A key requirement is the use of a shared secret key between the sender and receiver, which is essential for generating and verifying the MAC.

Detailed breakdown of the requirements are:

1. **Secure Key Sharing:** A secret key, known only to the sender and receiver, must be securely shared. This key is used in the MAC generation process. The key must be generated randomly and kept confidential to prevent unauthorized access or forgery.
2. **MAC Generation:** The MAC is generated using a cryptographic algorithm (like HMAC) and the shared secret key, along with the message itself. The MAC is a fixed-size value, which is appended to the message for transmission.
3. **Message Integrity:** The MAC must be able to detect any changes or alterations made to the message during transmission.
If the MAC generated by the receiver matches the MAC transmitted with the message, it verifies that the message hasn't been tampered with.
4. **Authenticity:** The MAC verifies the authenticity of the message, meaning it confirms that the message originated from the claimed sender. The receiver can verify the authenticity by recomputing the MAC using the shared secret key and the received message.
5. **Security Against Forgery:** It must be computationally infeasible for an attacker to forge a MAC for a message they didn't create, even if they know the MAC algorithm. The MAC algorithm should be designed to make it extremely difficult for an attacker to create a valid MAC for a modified message.
6. **One-Way Function:** The MAC algorithm should be a one-way function, meaning it's easy to compute the MAC but computationally difficult to reverse. This ensures that even if an attacker knows the message and the MAC, they cannot determine the secret key.

In essence, a MAC system should:

Guarantee message integrity: Ensuring the message hasn't been altered in transit.

Establish sender authenticity: Verifying the message originated from the claimed sender.

Prevent message forgery: Making it computationally infeasible for an attacker to create a valid MAC for a modified or forged message.

7.8 Security of MACs

The security of a Message Authentication Code (MAC) lies in its ability to prevent forgery by ensuring that the recipient can verify the authenticity and integrity of the message. A MAC uses a shared secret key to generate a cryptographic checksum of the message. A secure MAC should be computationally difficult to forge, meaning it should be hard for an adversary to create a valid MAC for a message without knowing the secret key.

Security Considerations:

Key Security: The security of the MAC depends entirely on the confidentiality of the shared secret key. If the key is compromised, an attacker could create their own MACs and forge messages.

Algorithm Strength: Using weak or outdated MAC algorithms can make the system vulnerable to attacks.

Key Management: Proper key management practices are essential for securing MACs. MACs provide a valuable layer of security by verifying the authenticity and integrity of messages during communication. However, the security of a MAC is entirely dependent on the security of the shared secret key and the strength of the MAC algorithm used.

7.9 HMAC

HMAC stands for Hash-based Message Authentication Code. It's a cryptographic technique used to verify the integrity and authenticity of a message by using a cryptographic hash function and a secret key. Both the sender and receiver must share the same secret key.

How HMAC Works:

Secret Key: A shared secret key is used by both the sender and receiver.

Hash Function: A cryptographic hash function (like SHA-256) is used to generate a hash of the message.

HMAC Generation: The sender calculates the HMAC by using the secret key and the hash function on the message.

Transmission: The sender sends both the original message and the calculated HMAC to the receiver.

Verification: The receiver calculates the HMAC on the received message using the shared secret key and the same hash function.

Comparison: The receiver compares the calculated HMAC with the received HMAC.

Authentication: If the HMACs match, it means the message is authentic and hasn't been tampered with during transit.

Benefits of HMAC:

Integrity: Ensures the message hasn't been altered during transmission.

Authentication: Verifies the source of the message by ensuring only someone with the secret key could have created the HMAC.

Efficiency: Uses a symmetric key, making it faster than asymmetric encryption methods.

Security: Provides strong security by using a hash function that's designed to be resistant to collisions.

Examples of HMAC Usage:

Cloud Storage Authentication:

Google Cloud uses HMAC keys to authenticate requests to the Cloud Storage XML API.

Docusign Connect:

Docusign uses HMAC to secure webhooks, ensuring the integrity and authenticity of sent messages.

Various APIs and Web Services:

HMAC is widely used in APIs and web services to verify the authenticity and integrity of requests.

7.10 Digital Signature: Introduction to Digital Signatures

A digital signature is a cryptographic technique used to authenticate and verify the integrity of digital information, similar to a handwritten signature on a paper document. It uses mathematical algorithms to create an encrypted "stamp" that proves a message or document originated from a specific sender and hasn't been altered.

In 1976, Whitfield Diffie and Martin Hellman first described the notion of a digital signature scheme, although they only conjectured that such schemes existed based on functions that are trapdoor one-way permutations.

Features of a Digital Signature

Digital Signatures have come to great use because of the incorporation of three main features.

1. Security : Digital signatures are legally enforceable and are highly secured because of its cryptographic algorithm. Documents signed using a digital signature can never be forged because the sender's identity is authenticated and verified before attaching it to the document.

2. **Time efficient** : Anyone can easily sign a document in minutes without any fear of tampering. The extra time can be utilized in improving the workflow.

3. **Cost efficient** : Digital signatures cut off the additional costs like paper printing, mailing, traveling etc. thus making it favorable for people to exchange documents from anywhere and at any time without spending a single penny

Categories of Digital Signatures

There are three categories of digital signatures or Digital Signature Certificates (DSCs) :

1. Class 1

Class 1 DSCs are not used for official or business documents. They are used for simple authentication of entities like an email or user IDs where

there is little risk of data compromise.

2. Class 2

Class 2 DSCs are used for e-filing government documents like Income Tax returns, Goods and Services Tax (GST) returns, where there is

minimal risk of data compromise. It confirms the identity of the signer against the original data that has already been saved in a database.

3. Class 3

Class 3 DSCs require the signer to physically present themselves before the Certifying Authority (CA) in order to verify their identity. This class

of DSCs provide the highest level of security and are widely used in the process of e-auctions, e-tendering, e-ticketing, court filings etc.

How to create Digital Signatures online?

There are numerous e-Signing portals which enables you to generate a digital signature certificate. XtraTrust is one of the most trusted websites where you can create your own digital signature and also buy digital signature certificates. The creation or issuance of a digital signature certificate is subject to fulfillment of any of these three requirements:

1. The applicant must physically appear in front of a Certifying Authority (CA) with their original documents that may be needed for identification and verification purposes.

2. The applicant can apply for a DSC online which is possible through Aadhar e-KYC. In this, the applicant does not need to produce any other documents.

3. A letter or certificate issued by the applicant's bank mentioning the requirement of a DSC can also help in getting a digital signature. Care must be taken to ensure that the letter or certificate is verified and certified by the bank officials.

7.10 Digital Signature Standards

Digital signature standards are a set of rules and algorithms used to create and verify digital signatures, ensuring the authenticity and integrity of electronic documents and data. The most well-known example is the Digital Signature Standard (DSS), a Federal Information Processing Standard (FIPS) developed by the National Institute of Standards and Technology (NIST).

What they are:

- **Algorithms:**

Digital signature standards specify the algorithms used to generate and verify digital signatures, ensuring a consistent and secure process.

- **Standards:**

These standards establish a framework for how digital signatures are created, verified, and used, including key management, hashing, and signature generation.

- **Purpose:**

They provide a way to electronically sign documents and data, ensuring the integrity and authenticity of the information.

Key components:

- **Digital Signature Algorithm (DSA):** The DSA is a key algorithm used in DSS for generating digital signatures.
- **Hashing:** A hash function is used to create a unique "fingerprint" of the data to be signed, which is then used in the signature generation process.
- **Public and Private Keys:** Digital signatures rely on a pair of cryptographic keys: a private key for generating the signature and a public key for verifying it.

Why they're important:

- **Authentication:** They verify the identity of the signer, ensuring that the document was indeed signed by the person claiming to have signed it.
- **Integrity:** They ensure that the data has not been altered after it was signed.
- **Non-repudiation:** They provide evidence that the signature was created by a specific individual, making it difficult for the signer to deny that they created the signature.

Example: Digital Signature Standard (DSS)

- **Development:**

DSS was developed by NIST and NSA to provide a secure and widely accepted way to generate and verify digital signatures.

- **Usage:**

It's used in a variety of applications, including e-commerce, secure communication, and electronic document signing.

Parul[®] University **NAAC** **A++**
Vadodara, Gujarat
GRADE



<https://paruluniversity.ac.in/>